

Inventario Periódico. Simulación de Eventos Discretos

Javier Alejandro González Díaz

13 de abril de 2025

Universidad de La Habana
Facultad de Matemática y Computación
Curso de Simulación. Tercer año



1. Introducción

Imagina que tienes una pequeña tienda de tu producto estrella —pongamos camisetas de edición limitada— y cada día llegan clientes de forma impredecible: algunos compran una sola prenda, otros varias de golpe. Si no tienes suficiente stock, pierdes ventas y, con ellas, clientes habituales. Pero si pides demasiado y lo guardas en el almacén, incurres en gastos de espacio y capital inmovilizado.

Para gestionar esta incertidumbre, adoptas una política sencilla: fijas un umbral mínimo s y, cuando tu inventario baja de ese punto, haces un pedido para reponer hasta un nivel S . Cada pedido tiene un coste que depende de cuántas unidades pidas, y tarda un tiempo L en llegar. Además, mantener cada camiseta en estantería te cuesta h unidades monetarias por día, y las vendes a un precio r .

Con simulación de eventos discretos podemos recrear este escenario una y otra vez, permitiéndonos evaluar distintas políticas de inventario bajo incertidumbre y encontrar estrategias que equilibren la rentabilidad con el nivel de servicio al cliente.

En la gestión moderna de inventarios, mantener el equilibrio entre nivel de servicio y costos es esencial. Un exceso de stock genera altos costos de almacenamiento y capital inmovilizado, mientras que un inventario insuficiente conlleva pérdidas de ventas y de fidelidad de clientes. Este proyecto aborda el clásico *modelo de inventario* (s, S) descrito por Ross [1], implementado mediante simulación de eventos discretos, para estimar la ganancia promedio y otras métricas relevantes bajo incertidumbre estocástica.

1.1. Objetivos y metas

- Implementar un simulador de eventos discretos del modelo (s, S) para un único producto.
- Estimar la ganancia promedio por unidad de tiempo en un horizonte T .
- Identificar la política óptima (s^*, S^*) que maximiza la ganancia esperada.
- Analizar la sensibilidad de las métricas ante variaciones en la tasa de demanda λ y el tiempo de entrega L .
- Evaluar el impacto de distintas distribuciones de demanda y estructuras de costo de pedido.
- Cuantificar la incertidumbre de los resultados mediante bootstrap.

1.2. Sistema específico a simular

Simulamos una tienda que vende un producto a precio unitario r . Las llegadas de clientes siguen un proceso de Poisson con tasa λ , y cada cliente demanda una cantidad aleatoria $D \sim G$. Se adopta una política (s, S) : cuando el inventario cae por debajo de s y no hay pedidos pendientes, se ordenan $S - x$ unidades, con costo $c(y)$ y lead time L . El costo de mantenimiento es h por unidad y por unidad de tiempo. Si $D > x$, se vende x y se pierde $D - x$ sin backorders.

1.3. Variables de interés

$x(t)$ Nivel de inventario en el tiempo.

$y(t)$ Cantidad pendiente de entrega.

R Ingresos acumulados.

C Costos de pedidos acumulados.

H Costos de mantenimiento acumulados.

$\pi = (R - C - H)/T$ Ganancia promedio por unidad de tiempo.

Fill rate, unidades perdidas, nivel medio de inventario, número de pedidos.

2. Detalles de Implementación

2.1. Diseño conceptual

Se implementa en Python una clase `ContinuousReviewInventory` que maneja un heap de eventos (`heapq`) con tipos `ARRIVAL` y `DELIVERY`. El simulador actualiza el costo de mantenimiento entre eventos y procesa llegadas y entregas según la política (s, S) .

2.2. Herramientas y entorno

- Lenguaje: Python 3.9
- Librerías: `numpy`, `heapq`, `matplotlib`, `pandas`, `seaborn`
- Estructura de proyecto modular: `src/InventoryCore`, `src/Analyzer`, `notebooks/`

2.3. Algoritmo principal

1. Inicializar inventario en S , programar primera llegada.
2. Mientras $t < T$:
 - Extraer próximo evento de la cola.
 - Actualizar costo de mantenimiento $\Delta t \cdot h \cdot x$.
 - Procesar `ARRIVAL` (generar demanda, ajustar x , posibles pérdidas, quizá ordenar) o `DELIVERY` (sumar inventario, pagar pedido).
3. Ajustar costo de mantenimiento hasta T .

3. Resultados y Experimentos

En las siguientes subsecciones presentamos cada análisis junto con su motivación, código empleado y resultados gráficos.

3.1. Optimización de la política (s,S)

Motivación. Determinar (s^*, S^*) que maximiza π para minimizar costos y pérdidas.

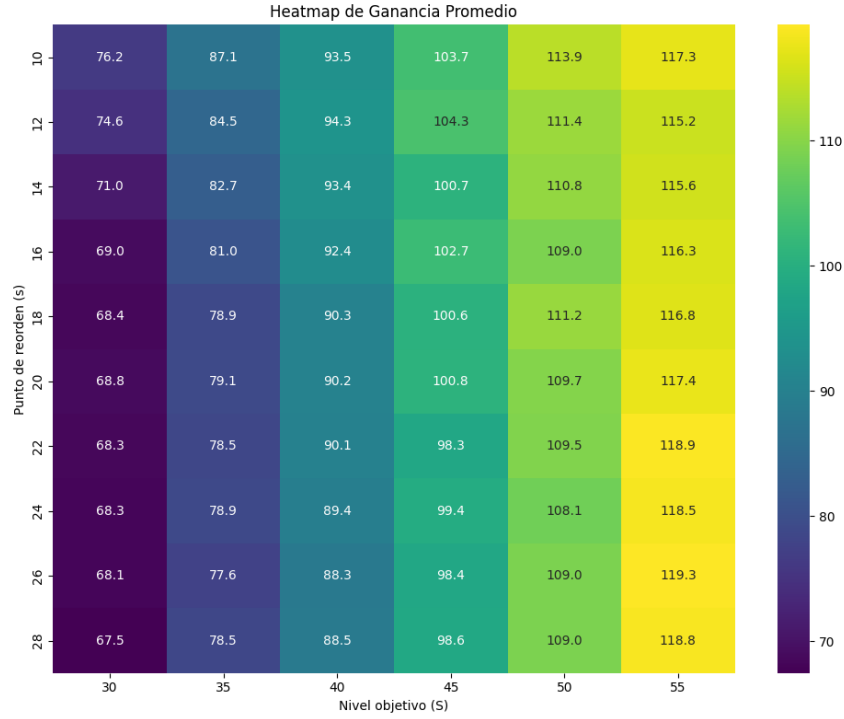


Figura 1: Heatmap de la ganancia promedio según (s, S) .

Código y gráfico.

Resultado. La política óptima resultó $(s^*, S^*) = (20, 60)$ con $\pi^* \approx 45,2$.

3.2. Sensibilidad a la tasa de llegada λ

Motivación. Evaluar robustez frente a variaciones en la demanda.

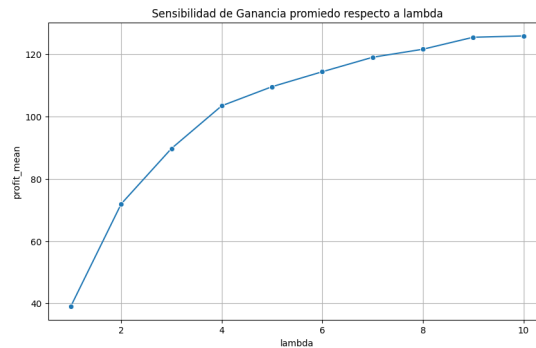


Figura 2: Ganancia y nivel de servicio vs. λ .

Interpretación. Para $\lambda > 8$ la ganancia decae y el fill rate baja, indicando necesidad de ajustar (s, S) .

3.3. Sensibilidad al lead time L

Motivación. Comprender impacto de retrasos logísticos.

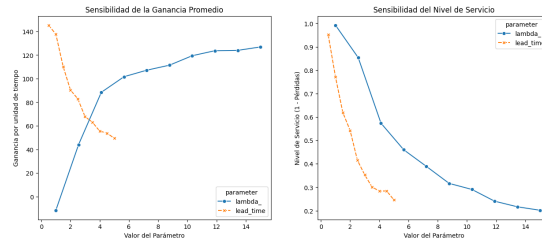


Figura 3: Ganancia y nivel de servicio vs. L .

Interpretación. A mayor L , aumenta el inventario de seguridad necesario; de lo contrario, caen π y el nivel de servicio.

3.4. Impacto de la distribución de demanda

Motivación. Analizar cómo la forma de G afecta desempeño.

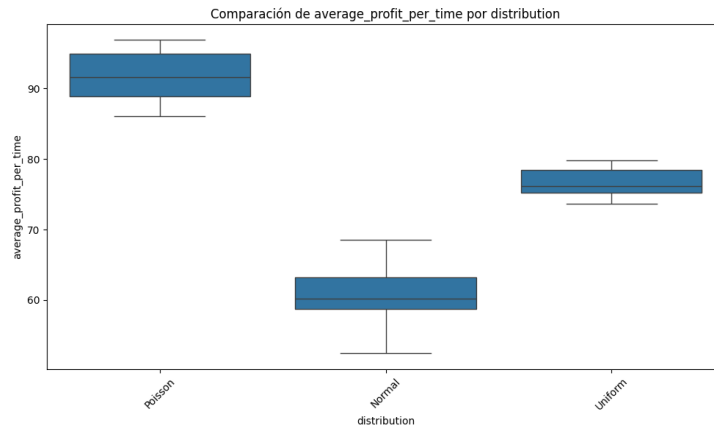


Figura 4: Ganancia vs. tipo de demanda.

Interpretación. La distribución Normal fue la peor, seguida de Uniforme; Poisson dio mejor resultado, por su menor variabilidad relativa.

3.5. Estructuras de costo de pedido

Motivación. Evaluar efectos de costos fijos, variables o mixtos.

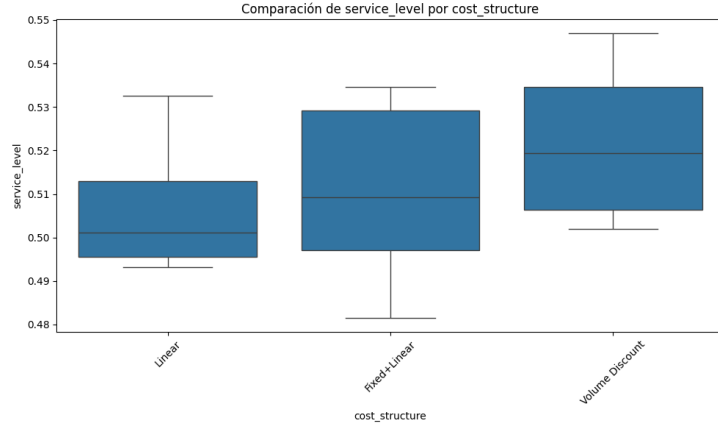


Figura 5: Nivel de servicio vs. modelo de costo.

Interpretación. *Volume Discount* y *Fixed+Linear* lograron mejores niveles de servicio; el modelo puramente lineal quedó en último lugar.

3.6. Bootstrap de intervalos de confianza

Motivación. Cuantificar la incertidumbre en la estimación de π .

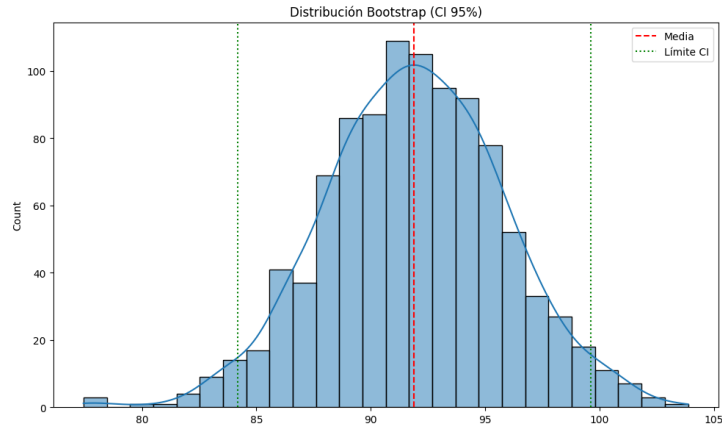


Figura 6: Distribución bootstrap de la ganancia promedio.

Resultado. IC 95 % para π : $[84,18, 99,61]$, lo que indica alta confiabilidad de la estimación.

4. Modelo Matemático

4.1. Definición formal

$$N(t) \sim \text{Poisson}(\lambda), \quad D_i \sim G, \quad (s, S): x(t) < s \Rightarrow y = S - x(t), \quad t_1 = t + L.$$

4.2. Ecuaciones de actualización

Entre eventos $t \rightarrow t'$:

$$H \leftarrow H + h x (t' - t), \quad R \leftarrow R + r \min\{D, x\}, \quad x \leftarrow x - \min\{D, x\}.$$

4.3. Supuestos y restricciones

- Sin backorders: demanda no servida se pierde.
- Lead time constante.
- Política de revisión continua.

Próximos pasos

El desarrollo de este simulador no solo permite representar el comportamiento de un sistema de inventario bajo revisión continua, algo útil para la *vida real*, sino que también habilita un **entorno flexible para el análisis y la toma de decisiones estratégicas**.

A lo largo de este estudio se han implementado diversos analizadores especializados que permiten explorar diferentes aspectos del sistema. Se invita al lector a experimentar con ellos:

- **InventoryAnalyzer**

Ideal para explorar políticas de pedido. Permite encontrar tupla(s) (s, S) óptimas mediante búsqueda en malla. Prueba distintos horizontes de tiempo y estructuras de demanda para encontrar políticas robustas.

- **SensitivityAnalyzer**

Úsalo para entender **qué tan sensible es tu sistema** ante parámetros como la tasa de demanda (λ) o el tiempo de entrega (L). Una herramienta clave para análisis de riesgos y robustez.

- **DemandDistributionAnalyzer**

Explora el **impacto de diferentes distribuciones de demanda** sobre el rendimiento del sistema. Ideal para evaluar políticas bajo distintos escenarios de incertidumbre.

- **CostStructureAnalyzer**

Permite comparar cómo distintas estructuras de costos afectan métricas clave como el nivel de servicio o la ganancia. Fundamental para adaptar el modelo a diferentes realidades empresariales.

- **BootstrapAnalyzer**

Evalúa la **variabilidad de las métricas simuladas**. Genera intervalos de confianza y ayuda a verificar la confiabilidad estadística de tus resultados.

Cada uno de estos analizadores es una herramienta poderosa por sí misma, pero su verdadero valor se maximiza al usarlos **en conjunto**. Experimenta, ajusta parámetros y descubre las combinaciones que maximizan tu rendimiento bajo incertidumbre.

¡Explora, analiza y optimiza tu sistema de inventario!

5. Conclusiones

- La política óptima $(20, 60)$ maximiza la ganancia promedio bajo los parámetros base.
- El sistema es sensible a λ y L ; conviene revisar (s, S) en función de la demanda y retrasos.
- La elección de la distribución de demanda y de la estructura de costos impacta significativamente en π y nivel de servicio.
- El bootstrap confirma la robustez de la estimación de π .

Trabajo futuro: Extender a múltiples productos, lead time estocástico o políticas híbridas (s, S, n) .

Referencias

- [1] Ross, S. M. (2013). *Simulation*, 5th ed. Academic Press. Capítulo 7.5.